

Struktury danych	
Kierunek <i>Informatyczne Systemy Automatyki</i>	Termin <i>poniedziałek TN 11¹⁵ – 13⁰⁰</i>
Imię, nazwisko, numer albumu <i>Jan Hulbój 284197, Maciej Polanowski 284195</i>	Data <i>13.04.2026</i>
Link do projektu https://github.com/mlody-jano/dataStructures	



Politechnika
Wrocławska

SPRAWOZDANIE – MINIPROJEKT NR 3

Poniższy dokument jest kompleksowym sprawozdaniem opisującym badania przeprowadzone w ramach trzeciego projektu na przedmiocie **Struktury Danych**. Obiektem trzeciego projektu była samodzielna implementacja **słownika**, z wykorzystaniem trzech różnych wariantów tablic mieszających, jednej uwzględniającej **drzewo AVL**. Każda ze struktur została zbadana pod kątem złożoności obliczeniowej wybranych operacji, które zostały podane w wymaganiach projektowych.

Spis treści

1 Wstęp teoretyczny	2
1.1 Tablica mieszająca z adresowaniem otwartym	2
1.2 Tablica mieszająca z kubelkami z listą	2
1.3 Tablica mieszająca z kubelkami ze zbalansowanym BST	3
2 Założenia projektowe	4
3 Wyniki badań	5
3.1 Operacja <code>insert()</code>	5
3.1.1 Zapełnienie tablicy 25%	5
3.1.2 Zapełnienie tablicy 50%	6
3.1.3 Zapełnienie tablicy 75%	7
3.1.4 Zapełnienie tablicy 100%	8
3.2 Operacja <code>remove()</code>	9
3.2.1 Zapełnienie tablicy 25%	9
3.2.2 Zapełnienie tablicy 50%	10
3.2.3 Zapełnienie tablicy 75%	11
3.2.4 Zapełnienie tablicy 100%	12
4 Wnioski	13

Spis rysunków

1 Średni czas operacji <code>insert()</code> przy zadanym zapełnieniu 25%	5
2 Średni czas operacji <code>insert()</code> przy zadanym zapełnieniu 50%	6
3 Średni czas operacji <code>insert()</code> przy zadanym zapełnieniu 75%	7
4 Średni czas operacji <code>insert()</code> przy zadanym zapełnieniu 100%	8
5 Średni czas operacji <code>remove()</code> przy zadanym zapełnieniu 25%	9
6 Średni czas operacji <code>remove()</code> przy zadanym zapełnieniu 50%	10
7 Średni czas operacji <code>remove()</code> przy zadanym zapełnieniu 75%	11
8 Średni czas operacji <code>remove()</code> przy zadanym zapełnieniu 100%	12

Spis tabel

1	Teoretyczne złożoności operacji dla badanych struktur	4
2	Średni czas operacji <code>insert()</code> przy zadanym zapelnieniu 25%	5
3	Średni czas operacji <code>insert()</code> przy zadanym zapelnieniu 50%	6
4	Średni czas operacji <code>insert()</code> przy zadanym zapelnieniu 75%	7
5	Średni czas operacji <code>insert()</code> przy zadanym zapelnieniu 100%	8
6	Średni czas operacji <code>remove()</code> przy zadanym zapelnieniu 25%	9
7	Średni czas operacji <code>remove()</code> przy zadanym zapelnieniu 50%	10
8	Średni czas operacji <code>remove()</code> przy zadanym zapelnieniu 75%	11
9	Średni czas operacji <code>remove()</code> przy zadanym zapelnieniu 100%	12

1 Wstęp teoretyczny

1.1 Tablica mieszająca z adresowaniem otwartym

Tablica mieszająca z adresowaniem otwartym przechowuje wszystkie elementy bezpośrednio w tablicy o rozmiarze m , bez użycia dodatkowych struktur pomocniczych. Operacja `insert(key, value)` oblicza indeks bazowy za pomocą funkcji mieszającej $h(key)$, a w przypadku kolizji (gdy obliczone miejsce jest już zajęte) stosuje tzw. **sondowanie** (ang. *probing*), przeszukując kolejne komórki tablicy zgodnie z wybraną strategią (np. sondowanie liniowe, kwadratowe lub podwójne mieszanie), aż do znalezienia wolnego miejsca. W zaimplementowanej wersji zastosowano sondowanie liniowe, czyli sprawdzanie kolejnych komórek tablicy z zawijaniem indeksu modulo rozmiar tablicy.

Operacja `remove(key)` wymaga specjalnego oznaczenia usuniętego elementu tzw. **znacznikiem nagrobkowym** (ang. *tombstone*), aby nie przerywać łańcuchów sondowania dla późniejszych operacji wyszukiwania.

Kluczowym parametrem wpływającym na wydajność tej struktury jest **współczynnik zapelnienia** (ang. *load factor*), który opisany jest następującym wzorem:

$$\alpha = \frac{n}{m} \quad (1)$$

gdzie n odpowiada ilości elementów w tablicy, a m odpowiada wielkości struktury. Wraz ze wzrostem α rośnie prawdopodobieństwo kolizji, co wydłuża łańcuchy sondowania i pogarsza złożoność operacji, w pesymistycznym przypadku ($\alpha \simeq 1$) wszystkie operacje degradują się do $O(n)$. W praktyce tablicę utrzymuje się przy $\alpha < 0,6$, co zapewnia średnią złożoność $O(1)$ dla `insert()` i `remove()`.

Rzeczywiste zastosowania:

- **Cache wyników obliczeń i szybkie mapowanie identyfikatorów:** Tablica mieszająca z adresowaniem otwartym dobrze sprawdza się w sytuacjach, w których najważniejszy jest bardzo szybki dostęp do danych oraz dobra lokalność pamięci. Przykładem może być cache, w którym kluczem jest argument funkcji, a wartością ostatnio obliczony wynik. Dzięki przechowywaniu elementów bezpośrednio w jednej tablicy struktura ogranicza liczbę dodatkowych alokacji pamięci i pozwala uzyskać średni czas operacji `insert()`, `find()` oraz `remove()` równy $O(1)$, o ile współczynnik zapelnienia jest utrzymywany na rozsądnym poziomie.
- **Tablice symboli i mapowanie krótkotrwałych identyfikatorów:** Adresowanie otwarte może być wykorzystywane w tablicach symboli, np. podczas działania kompilatora lub interpretera, gdzie nazwy zmiennych, tokeny albo identyfikatory obiektów są szybko mapowane na odpowiadające im informacje. W takim zastosowaniu struktura jest korzystna, ponieważ przy dobrze dobranej funkcji mieszającej i automatycznym zwiększaniu rozmiaru tablicy zapewnia szybkie operacje dostępu bez konieczności tworzenia dodatkowych list lub drzew w kubekach.

1.2 Tablica mieszająca z kubkami z listą

Tablica mieszająca z kubkami z listą (ang. *separate chaining*) rozwiązuje kolizje poprzez przechowywanie w każdej komórce tablicy wskaźnika na listę jednokierunkową (kubek), zawierającą wszystkie elementy o tym samym indeksie mieszającym. Operacja `insert(key, value)` oblicza indeks kubka,

a następnie przeszukuje listę w celu sprawdzenia, czy dany klucz już istnieje. Jeśli klucz zostanie odnaleziony, aktualizowana jest wartość. W przeciwnym przypadku nowy węzeł dodawany jest na początek listy. Średni koszt operacji zależy od długości listy w kubelku.

Operacja `find(key)` wymaga przejścia przez listę w obliczonym kubelku, aż do znalezienia szukanego klucza, w przypadku równomiernego rozkładu kluczy średnia długość listy wynosi $\alpha = \frac{n}{m}$, zapewniając średnią złożoność $O(1 + \alpha)$. Operacja `remove(key)` przebiega analogicznie do `find()`, z dodatkowym usunięciem węzła z listy.

W odróżnieniu od adresowania otwartego, współczynnik zapelnienia α może przekraczać 1 bez degradacji struktury do przypadku pesymistycznego, jednak zbyt duże wypełnienie wydłuża listy i pogarsza wydajność wyszukiwania. Struktura jest odporna na skupiska kolizji (ang. *clustering*), które stanowią główną wadę adresowania otwartego.

Rzeczywiste zastosowania:

- **Proste słowniki o zmiennej liczbie elementów:** Tablica mieszająca z kubkami w postaci list jednokierunkowych dobrze sprawdza się jako prosta implementacja słownika, w którym liczba elementów może przekroczyć początkową pojemność tablicy. Przykładem może być mapa przechowująca dane studentów, gdzie kluczem jest numer indeksu, a wartością rekord z informacjami o studencie. W przypadku kolizji elementy są dopisywane do listy w odpowiednim kubku, dzięki czemu struktura nie wymaga stosowania znaczników usunięcia ani natychmiastowego powiększania tablicy.
- **Systemy magazynowe i ewidencja obiektów:** Wariant z listami może być używany w systemach, w których istotna jest prostota implementacji i łatwa obsługa kolizji, np. w mapie kod produktu - liczba sztuk w magazynie. Nawet jeśli kilka produktów trafi do tego samego kubka, mogą one zostać przechowane w liście. Przy równomiernym rozkładzie kluczy średnia długość listy pozostaje niewielka, dlatego operacje `insert()` oraz `remove()` mają w praktyce niski koszt.

1.3 Tablica mieszająca z kubkami ze zbalansowanym BST

Tablica mieszająca z kubkami ze zbalansowanym drzewem BST (w tym projekcie: drzewem AVL) różni się od wariantu z listą sposobem organizacji elementów wewnątrz każdego kubka. Zamiast listy jednokierunkowej, każdy kubek przechowuje drzewo AVL, samobalansujące drzewo binarne poszukiwań, które po każdej operacji `insert()` lub `remove()` wykonuje lokalne rotacje w celu utrzymania własności AVL: różnica wysokości lewego i prawego poddrzewa każdego węzła nie przekracza 1. Dzięki temu wysokość drzewa AVL wynosi zawsze $O(\log k)$, gdzie k to liczba elementów w kubku.

Operacje `insert(key, value)`, `find(key)` i `remove(key)` polegają najpierw na obliczeniu indeksu tablicy przez funkcję mieszającą, a następnie na wykonaniu odpowiedniej operacji na drzewie AVL w danym kubku. W przypadku równomiernego rozkładu kluczy i niskiego współczynnika zapelnienia α , drzewa AVL w kubkach zawierają przeciętnie po jednym elemencie, przez co koszt operacji jest taki sam jak w wariancie z listą - $O(1)$.

Przewaga drzewa AVL ujawnia się przy wysokich wartościach α lub patologicznych funkcjach mieszających, gdy wiele kluczy trafia do jednego kubka: zamiast liniowego przeszukiwania listy $O(k)$, operacja na drzewie AVL zajmuje jedynie $O(\log k)$.

Rzeczywiste zastosowania:

- **Słowniki odporne na niekorzystny rozkład kluczy:** Tablica mieszająca z kubkami opartymi na drzewach AVL może być stosowana tam, gdzie istnieje ryzyko dużej liczby kolizji w pojedynczych kubkach. Przykładem może być system przechowujący identyfikatory obiektów, w którym dane wejściowe mogą być nierównomiernie rozłożone lub częściowo zależne od użytkownika. W takim przypadku zastąpienie listy drzewem AVL ogranicza koszt operacji w kubku z $O(k)$ do $O(\log k)$, gdzie k oznacza liczbę elementów w danym kubku.
- **Systemy wymagające ograniczenia przypadku pesymistycznego:** Wariant z drzewami AVL jest użyteczny w systemach, w których ważne jest ograniczenie czasu działania nawet dla niekorzystnych danych. Przykładem może być struktura zliczająca wystąpienia żądań sieciowych, gdzie kluczem jest skrót lub identyfikator żądania, a wartością licznik wystąpień. Jeśli wiele kluczy trafia do tego samego kubka, drzewo AVL pozwala uniknąć liniowego przeszukiwania długiej listy i zapewnia logarytmiczny koszt operacji wewnątrz kubka.

Tabela 1: Teoretyczne złożoności operacji dla badanych struktur

Struktura	Złożoność operacji		
	<code>insert()</code> średnio	<code>remove()</code> średnio	<code>insert/remove</code> pesymist.
Adresowanie otwarte	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Kubelki z listą	$\mathcal{O}(1 + \alpha)$	$\mathcal{O}(1 + \alpha)$	$\mathcal{O}(n)$
Kubelki z drzewem AVL	$\mathcal{O}(\log k)$	$\mathcal{O}(\log k)$	$\mathcal{O}(\log n)$

gdzie:

$\alpha = n/m$ – współczynnik zapelnienia tablicy,

k – liczba elementów w pojedynczym kubelku.

2 Założenia projektowe

W celu zapewnienia odpowiedniego podłoża merytorycznego do wyciągnięcia wniosków dotyczących badanych struktur danych, przyjęte zostały następujące założenia:

- **Rozmiary struktur:** 10 rozmiarów z zakresu 10000 - 1000000, skok co 100 tys.
- **Ilość powtórzeń operacji:** 100
- **Sposób generacji danych:** Skrypt Python generujący dane do pliku `.csv`.
- **Sposób pomiaru czasu:** Uśrednienie czasu operacji dla wybranej liczby kopii.

Testy utworzonych struktur danych były prowadzone na urządzeniu wyposażonym w następujące komponenty:

- **Procesor** - Apple M4 10 Core CPU 4Ghz
- **RAM** - 16GB

3 Wyniki badań

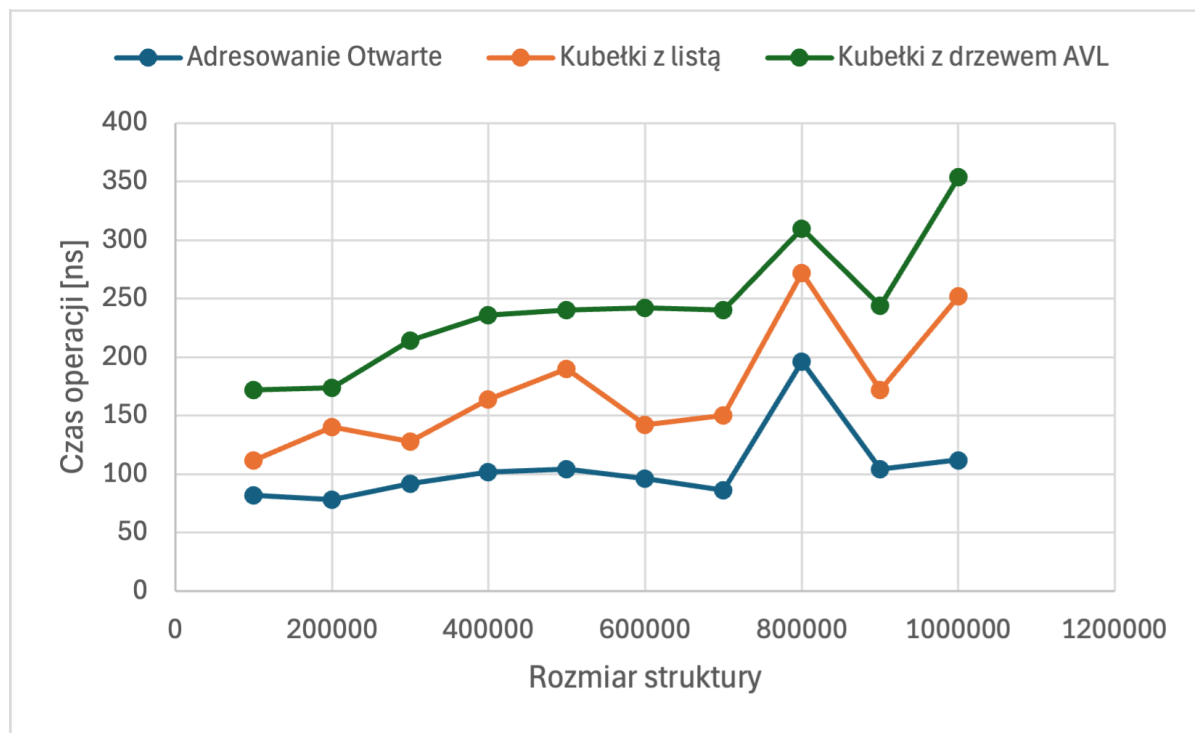
3.1 Operacja insert()

W implementacji z adresowaniem otwartym zastosowano automatyczne zwiększanie rozmiaru tablicy po przekroczeniu ustalonego progu zapelnienia. Oznacza to, że dla wysokich zadanych poziomów zapelnienia, takich jak 75% i 100%, rzeczywista pojemność tablicy w momencie pomiaru mogła być większa od pojemności początkowej.

3.1.1 Zapelnienie tablicy 25%

Tabela 2: Średni czas operacji insert() przy zadanym zapelnieniu 25%

Lp.	Capacity	Czas operacji [ns]		
		OpenAddressing	HashTableList	AVL
1	100000	82	112	172
2	200000	78	140	174
3	300000	92	128	214
4	400000	102	164	236
5	500000	104	190	240
6	600000	96	142	242
7	700000	86	150	240
8	800000	196	272	310
9	900000	104	172	244
10	1000000	112	252	354

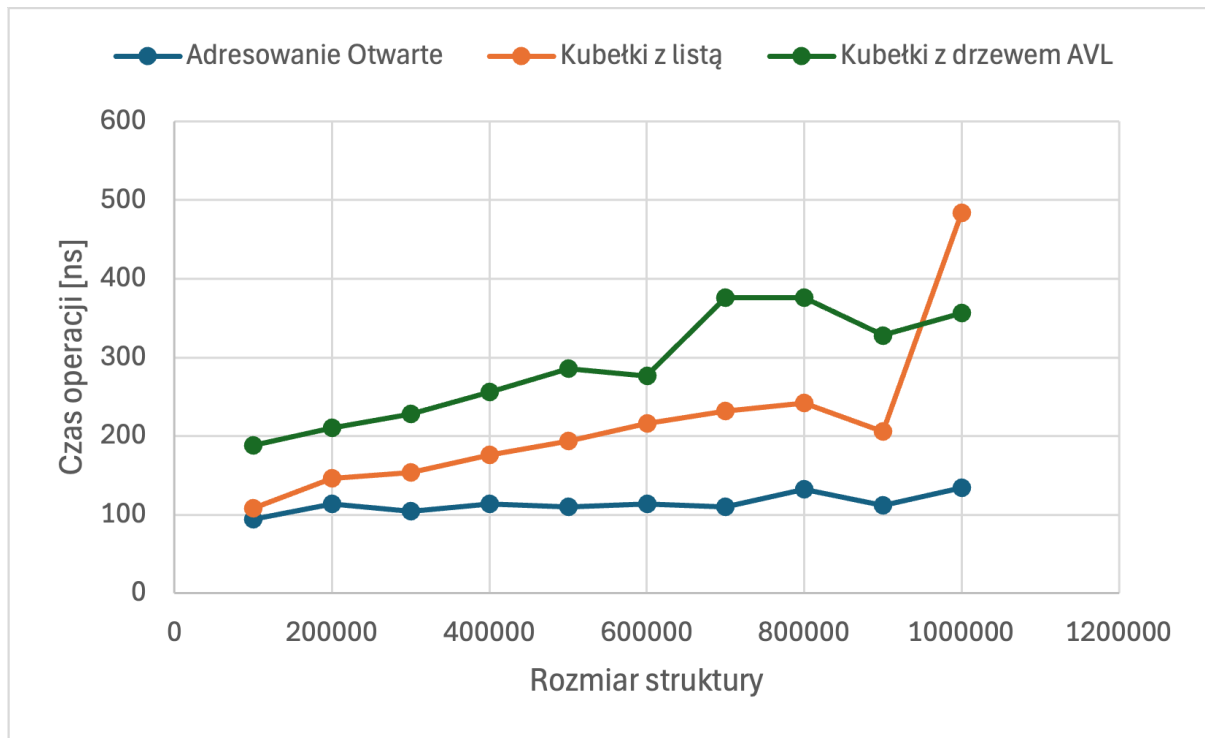


Rysunek 1: Średni czas operacji insert() przy zadanym zapelnieniu 25%

3.1.2 Zapelnienie tablicy 50%

Tabela 3: Średni czas operacji `insert()` przy zadanym zapelnieniu 50%

Lp.	Capacity	Czas operacji [ns]		
		OpenAddressing	HashTableList	AVL
1	100000	94	108	188
2	200000	114	146	210
3	300000	104	154	228
4	400000	114	176	256
5	500000	110	194	286
6	600000	114	216	276
7	700000	110	232	376
8	800000	132	242	376
9	900000	112	206	328
10	1000000	134	484	356

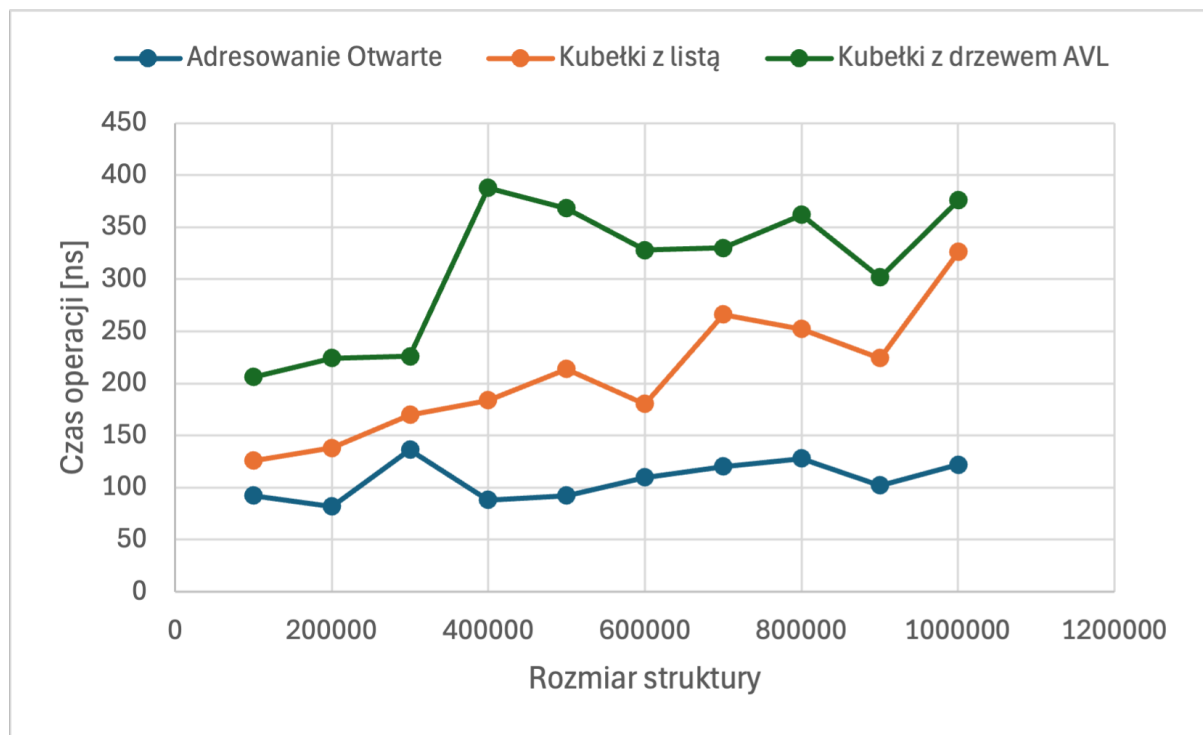


Rysunek 2: Średni czas operacji `insert()` przy zadanym zapelnieniu 50%

3.1.3 Zapelnienie tablicy 75%

Tabela 4: Średni czas operacji `insert()` przy zadanym zapelnieniu 75%

Lp.	Capacity	Czas operacji [ns]		
		OpenAddressing	HashTableList	AVL
1	100000	92	126	206
2	200000	82	138	224
3	300000	136	170	226
4	400000	88	184	388
5	500000	92	214	368
6	600000	110	180	328
7	700000	120	266	330
8	800000	128	252	362
9	900000	102	224	302
10	1000000	122	326	376

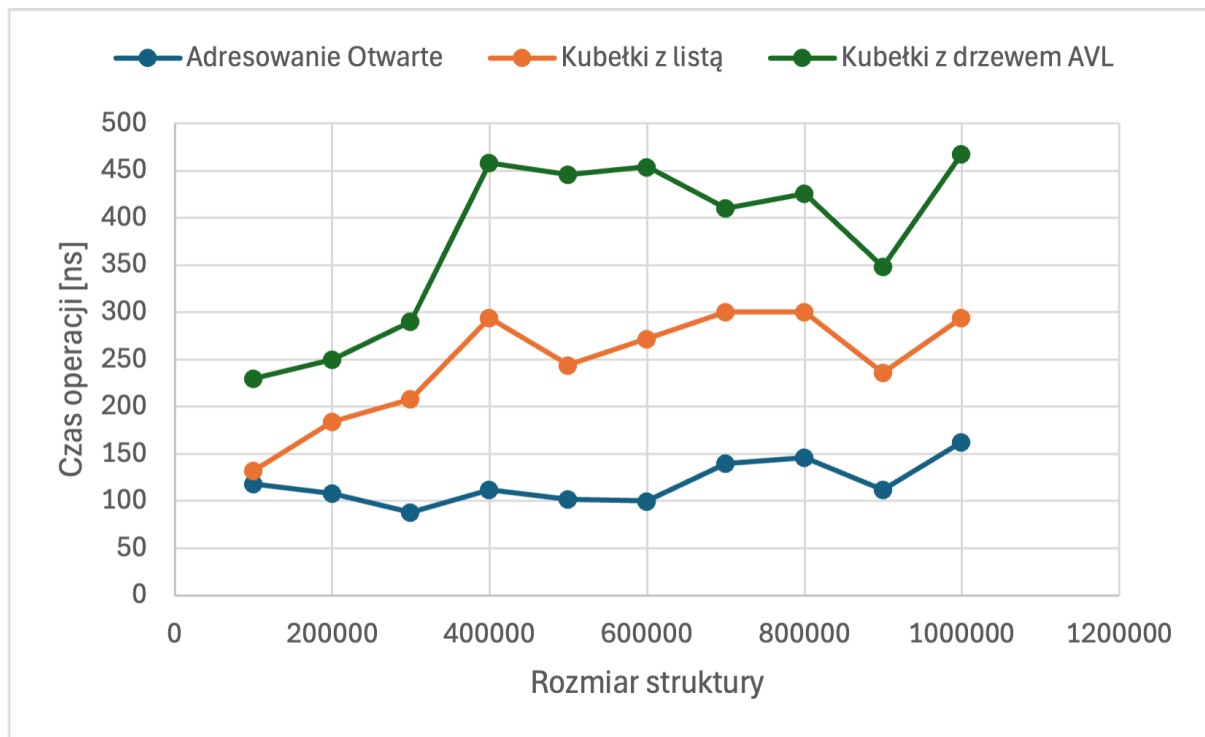


Rysunek 3: Średni czas operacji `insert()` przy zadanym zapelnieniu 75%

Tabela 5: Średni czas operacji `insert()` przy zadanym zapelnieniu 100%

Lp.	Capacity	Czas operacji [ns]		
		OpenAddressing	HashTableList	AVL
1	100000	118	132	230
2	200000	108	184	250
3	300000	88	208	290
4	400000	112	294	458
5	500000	102	244	446
6	600000	100	272	454
7	700000	140	300	410
8	800000	146	300	426
9	900000	112	236	348
10	1000000	162	294	468

3.1.4 Zapelnienie tablicy 100%



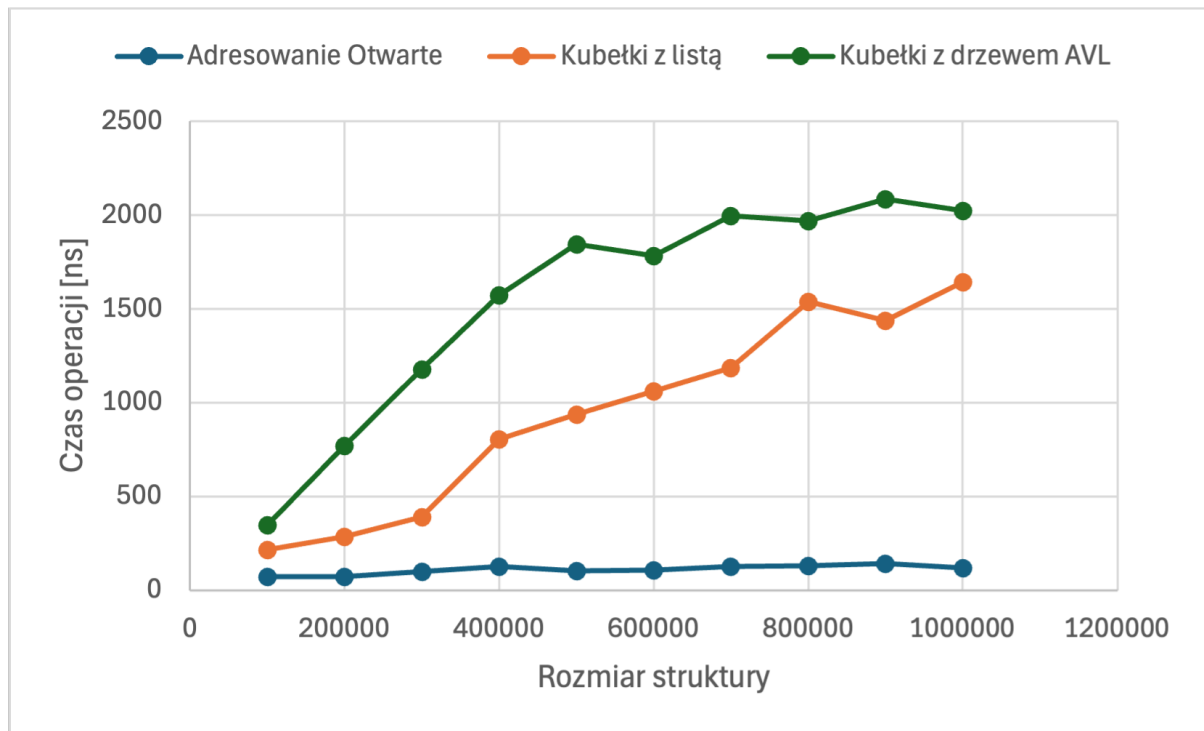
Rysunek 4: Średni czas operacji `insert()` przy zadanym zapelnieniu 100%

3.2 Operacja remove()

3.2.1 Zapełnienie tablicy 25%

Tabela 6: Średni czas operacji remove() przy zadanym zapełnieniu 25%

Lp.	Capacity	Czas operacji [ns]		
		OpenAddressing	HashTableList	AVL
1	100000	74	216	348
2	200000	72	286	772
3	300000	100	390	1178
4	400000	126	804	1574
5	500000	104	936	1844
6	600000	106	1062	1784
7	700000	128	1186	1996
8	800000	132	1538	1970
9	900000	144	1436	2086
10	1000000	120	1642	2022

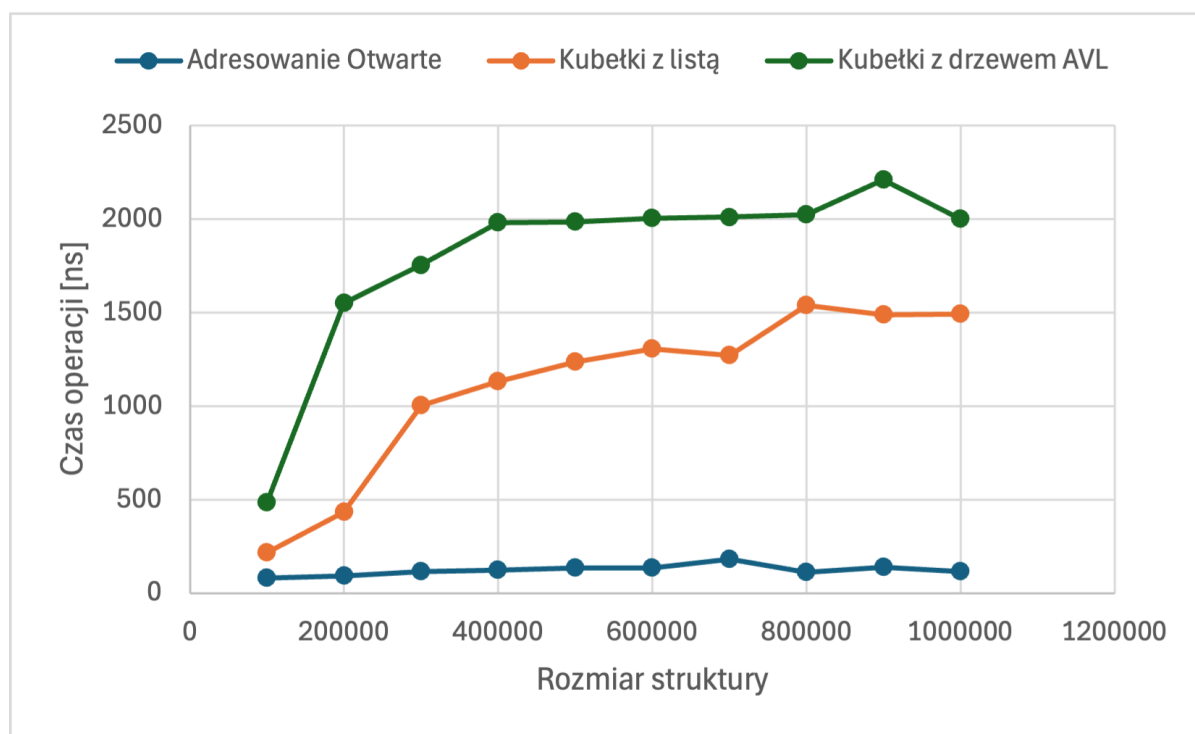


Rysunek 5: Średni czas operacji remove() przy zadanym zapełnieniu 25%

Tabela 7: Średni czas operacji `remove()` przy zadanym zapelnieniu 50%

Lp.	Capacity	Czas operacji [ns]		
		OpenAddressing	HashTableList	AVL
1	100000	82	216	482
2	200000	92	434	1550
3	300000	114	1004	1754
4	400000	122	1130	1982
5	500000	134	1236	1986
6	600000	134	1306	2004
7	700000	180	1272	2010
8	800000	110	1540	2024
9	900000	138	1488	2212
10	1000000	116	1492	2000

3.2.2 Zapelnienie tablicy 50%

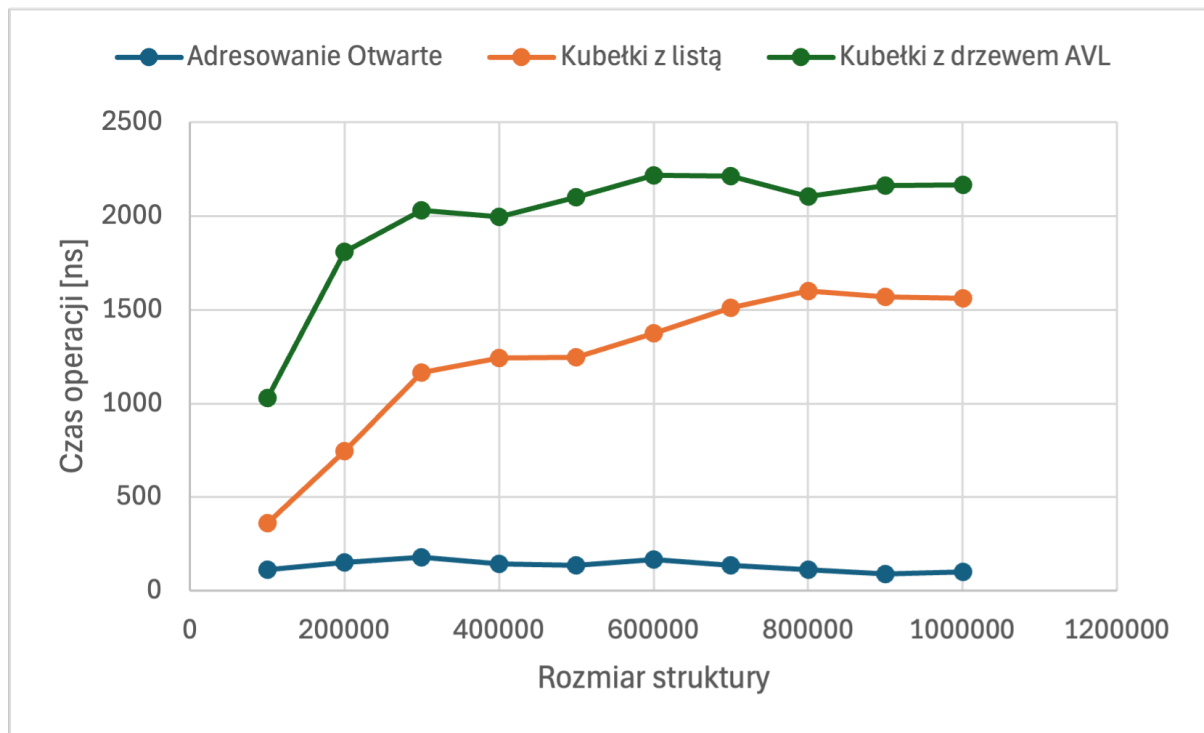


Rysunek 6: Średni czas operacji `remove()` przy zadanym zapelnieniu 50%

3.2.3 Zapęlnienie tablicy 75%

Tabela 8: Średni czas operacji `remove()` przy zadanym zapęlnieniu 75%

Lp.	Capacity	Czas operacji [ns]		
		OpenAddressing	HashTableList	AVL
1	100000	114	364	1030
2	200000	154	748	1810
3	300000	180	1164	2030
4	400000	146	1242	1996
5	500000	138	1248	2100
6	600000	168	1374	2218
7	700000	138	1512	2214
8	800000	114	1602	2104
9	900000	90	1570	2164
10	1000000	104	1562	2166

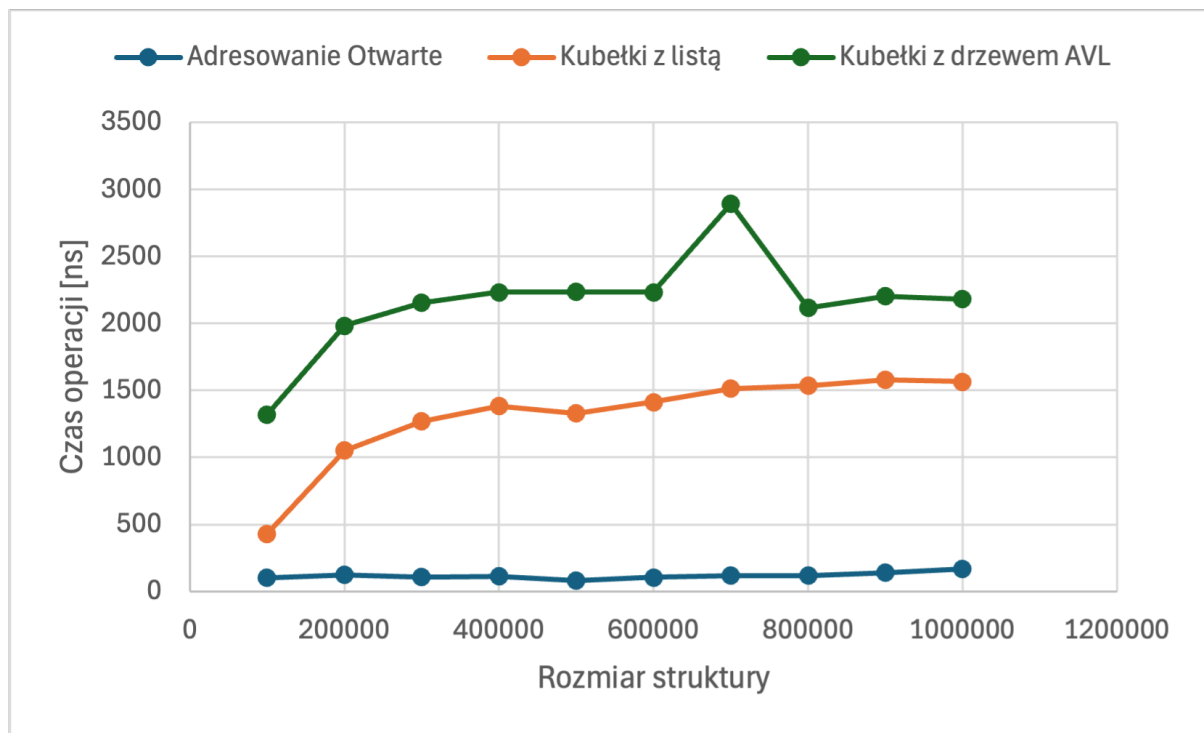


Rysunek 7: Średni czas operacji `remove()` przy zadanym zapęlnieniu 75%

3.2.4 Zapełnienie tablicy 100%

Tabela 9: Średni czas operacji `remove()` przy zadanym zapełnieniu 100%

Lp.	Capacity	Czas operacji [ns]		
		OpenAddressing	HashTableList	AVL
1	100000	102	428	1320
2	200000	124	1052	1984
3	300000	108	1270	2154
4	400000	114	1384	2234
5	500000	80	1330	2238
6	600000	104	1414	2234
7	700000	116	1514	2896
8	800000	120	1534	2116
9	900000	140	1578	2202
10	1000000	166	1566	2184



Rysunek 8: Średni czas operacji `remove()` przy zadanym zapełnieniu 100%

4 Wnioski

Na podstawie przeprowadzonych badań można zauważyć wyraźne różnice pomiędzy trzema zaimplementowanymi wariantami tablic mieszających. Najlepsze wyniki czasowe w większości przypadków uzyskala tablica mieszająca z adresowaniem otwartym. Wynika to przede wszystkim z faktu, że wszystkie elementy są przechowywane bezpośrednio w jednej tablicy, co zapewnia dobrą lokalność pamięci i ogranicza liczbę dodatkowych alokacji dynamicznych. Dla operacji `insert()` czasy tej struktury pozostawały stosunkowo niskie i stabilne dla wszystkich badanych pojemności.

Należy jednak zaznaczyć, że w implementacji z adresowaniem otwartym zastosowano automatyczne zwiększanie rozmiaru tablicy po przekroczeniu określonego progu zapelnienia. Z tego powodu dla wysokich zadanych poziomów zapelnienia, szczególnie 75% oraz 100%, rzeczywista pojemność tablicy w momencie wykonywania pomiaru mogła być większa od pojemności początkowej. Oznacza to, że bardzo dobre wyniki tej struktury przy wysokich poziomach zapelnienia wynikają częściowo z mechanizmu przebudowy tablicy, który zapobiega pracy przy rzeczywistym zapelnieniu bliskim 1.

Tablica mieszająca z kubelkami w postaci list jednokierunkowych uzyskała wyniki gorsze niż adresowanie otwarte, szczególnie dla operacji `remove()`. Jest to zgodne z charakterem tej struktury, ponieważ odnalezienie oraz usunięcie elementu wymaga przejścia przez listę znajdującą się w odpowiednim kubelku. Wraz ze wzrostem liczby elementów rośnie koszt przeszukiwania kubelków, a dodatkowy wpływ mają alokacje dynamiczne i gorsza lokalność pamięci w porównaniu z adresowaniem otwartym.

Wariant z kubelkami opartymi na drzewach AVL okazał się najwolniejszy w większości przeprowadzonych testów, mimo że teoretycznie zapewnia lepszą złożoność operacji wewnątrz pojedynczego kubelka. Wynika to z faktu, że przy równomiernym rozkładzie kluczy liczba elementów w pojedynczym kubelku nie jest na tyle duża, aby przewaga złożoności $O(\log k)$ zrównoważyła narzut związany z obsługą drzewa AVL. Każda operacja na drzewie wymaga dodatkowych porównań, aktualizacji wysokości węzłów oraz potencjalnych rotacji, co zwiększa rzeczywisty czas działania.

Wyniki pokazują, że dla danych losowych i względnie równomiernego rozkładu kluczy najbardziej efektywną praktycznie strukturą okazała się tablica mieszająca z adresowaniem otwartym. Wariant z listami jest prostszy implementacyjnie i nie wymaga specjalnych znaczników usunięcia, ale jego wydajność pogarsza się wraz ze wzrostem długości list w kubelkach. Wariant z AVL ma największy sens w przypadkach, gdy można spodziewać się dużej liczby kolizji i długich kubelków, np. przy niekorzystnej funkcji mieszającej lub celowo kolizyjnych danych wejściowych. W badanych warunkach taki przypadek nie wystąpił w stopniu wystarczającym, dlatego narzut drzewa AVL był większy niż korzyść z jego zastosowania.

Podsumowując, przeprowadzone badania potwierdzają, że sama teoretyczna złożoność operacji nie zawsze przekłada się bezpośrednio na najlepszy wynik czasowy. W praktyce bardzo duże znaczenie mają również lokalność pamięci, liczba alokacji dynamicznych, koszt obsługi struktur pomocniczych oraz sposób zarządzania zapelnieniem tablicy.